



MEDIACREST
TRAINING COLLEGE

Machine Learning I: Supervised Learning (Regression Focus)

From Statistical Models to Predictive ML Pipelines for Real Estate Valuation

Victor Wandera Lumumba, MSc.

Statistician | Data Analyst | ML Expert

Mediacrest Training College

2nd September 2026

Abstract

This presentation bridges the gap between traditional statistical modeling and modern machine learning regression, utilizing a real estate dataset to demonstrate robust predictive frameworks.

- **Foundations:** Robust data preparation, strategic feature engineering, and leak-proof ML pipeline design.
- **Algorithms:** Deep dive into Regularization (Ridge, Lasso, Elastic Net) and Tree-based models (Decision Trees, Random Forest, Gradient Boosting).
- **Evaluation:** Rigorous cross-validation, hyperparameter tuning (GridSearchCV), and test set evaluation using RMSE, MAE, and R^2 .
- **Application:** Real-world revenue forecasting and property valuation with actionable business and policy insights.

Session Objectives

- Move from statistical models to robust machine learning regression pipelines.
- Control model complexity and prevent overfitting using regularization techniques.
- Select models based on both predictive performance metrics and interpretability.
- Master hyperparameter tuning via GridSearchCV and RandomizedSearchCV.
- Apply cross-validation strategies to ensure reliable generalization to unseen data.
- Translate model outputs and feature importances into actionable business recommendations.

Part 1: Data Preparation & Exploratory Analysis

"Understanding the data before building the pipeline."

Dataset & Dependencies

- **Dataset:** `Housing.csv` (Real estate valuation / revenue forecasting data).
- **Target Variable:** `price` (Continuous).
- **Features:** `area`, `bedrooms`, `bathrooms`, `stories`, `parking`, `mainroad`, `guestroom`, `basement`, `hotwaterheating`, `airconditioning`, `prefarea`, `furnishingstatus`.
- **Dependencies:**
 - Data Manipulation: `pandas`, `numpy`
 - Machine Learning: `scikit-learn`
 - Visualization: `matplotlib`, `seaborn`

1.1 Loading the Data

- Real estate datasets often contain inconsistent missing-value representations or require explicit type casting.
- **Best Practice:** Explicitly define missing value strings during data ingestion and set professional plotting styles.

Python Implementation

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import warnings
6 warnings.filterwarnings("ignore")
7
8 # Set professional plotting style
9 sns.set_theme(style="whitegrid", context="notebook")
10 plt.rcParams['figure.figsize'] = (12, 6)
11
12 # Load Data, explicitly handling missing values
13 df = pd.read_csv('Housing.csv', na_values=['NA', '?', 'NaN', ''])
```

1.2 Exploratory Data Analysis (EDA)

- **Data Inspection:** Check data types, shape, and missing value counts using `df.info()` and `df.isnull().sum()`.
- **Feature Categorization:** Clearly separate numeric features (e.g., area, bedrooms) from categorical features (e.g., mainroad, furnishingstatus).
- **Target Leakage Check:** Ensure no post-valuation variables (e.g., final sale date or post-inspection repairs) are used to predict the initial listing price.

Best Practice

Always separate your target variable (y) from your feature matrix (X) before any exploratory visualization to prevent accidental data leakage.

1.3 Target Variable Distribution

- Visualizing the price distribution using histograms and Kernel Density Estimation (KDE).
- **Observation:** Real estate prices are often right-skewed, with a long tail of high-value properties.
- **Action:** In advanced pipelines, a log-transformation (`np.log1p`) of the target variable may be applied to normalize the distribution and improve linear model performance.

Note on Evaluation

If the target is log-transformed during training, predictions must be inverse-transformed (`np.expm1`) before calculating RMSE or MAE to maintain interpretability in original currency units.

Part 2: Feature Engineering & ML Pipeline Design

"Building robust, leak-proof preprocessing workflows."

2.1 The ML Pipeline Concept

Why Use Pipelines?

- **Prevents Data Leakage:** Ensures preprocessing steps (like scaling or imputation) are fit **ONLY** on the training data during cross-validation, then applied to the validation fold.
- **Reproducibility:** Bundles transformers and the final estimator into a single, cohesive object.
- **Cleaner Code:** Eliminates the need to manually transform `X_train` and `X_test` at every step.

2.2 Preprocessing: Numeric vs Categorical

Numeric Features

StandardScaler

Transforms data to have a mean of 0 and a variance of 1.

Crucial for: Distance-based algorithms and Regularization (Ridge/Lasso/Elastic Net).

Categorical Features

OneHotEncoder

Converts categories into binary columns.

Crucial setting: `drop='first'` to avoid the dummy variable trap (perfect multicollinearity) when an intercept is present.

2.3 ColumnTransformer & Pipeline Setup

Python Implementation

```
1 from sklearn.pipeline import Pipeline
2 from sklearn.compose import ColumnTransformer
3 from sklearn.preprocessing import StandardScaler, OneHotEncoder
4
5 numeric_features = ['area', 'bedrooms', 'bathrooms', 'stories', 'parking']
6 categorical_features = ['mainroad', 'guestroom', 'basement', 'hotwaterheating',
7                        'airconditioning', 'prefarea', 'furnishingstatus']
8
9 preprocessor = ColumnTransformer(
10     transformers=[
11         ('num', StandardScaler(), numeric_features),
12         ('cat', OneHotEncoder(drop='first', handle_unknown='ignore'),
13          categorical_features)
14     ])
15
16 # The pipeline will later be: Pipeline([('preprocessor', preprocessor), ('model',
17                                         Model())])
```

2.4 Train / Test Split Strategy

- **Goal:** Evaluate the model's ability to generalize to completely unseen data.
- **Implementation:** 80% Training, 20% Testing.
- **Reproducibility:** `random_state=42` ensures the split is identical across runs.

Python Implementation

```
1 from sklearn.model_selection import train_test_split
2
3 X = df.drop('price', axis=1)
4 y = df['price']
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.20, random_state=42
8 )
9
```

2.5 Why Feature Scaling is Crucial

The Regularization Trap

Regularization penalties (L1 and L2) are **distance-based**.

- If area is measured in thousands (e.g., 5000 sq ft) and bedrooms in single digits (e.g., 3), the penalty will disproportionately shrink the coefficient of area simply because of its larger magnitude, not its predictive importance.
- **Solution:** Standardizing ensures all features contribute equally to the penalty term, allowing the algorithm to select features based on true predictive power.

Part 3: Model Definition & Cross-Validation

"Evaluating generalization before touching the test set." is the key to robust ML.

3.1 Algorithms Covered

Linear Models (Regularized)

- **Ridge (L2):** Shrinks coefficients, handles multicollinearity.
- **Lasso (L1):** Shrinks coefficients to exactly zero (feature selection).
- **Elastic Net:** Combines L1 and L2 penalties.

Non-Linear Tree Models

- **Decision Tree:** Captures non-linear thresholds, prone to overfitting.
- **Random Forest:** Ensemble of trees, reduces variance via bagging.
- **Gradient Boosting:** Sequentially corrects errors (e.g., XGBoost).

- **Purpose:** Provides a robust estimate of model performance by training and validating on different subsets of the training data, preventing overfitting to a single train/val split.

- **Implementation:** 5-Fold Cross-Validation.

`cv = KFold(n_splits = 5, shuffle = True, random_state = 42)`

Note: Scikit-learn returns negative MSE, so we negate and sqrt for RMSE

`rmse_scores = np.sqrt(-cross_val_score(model, X_train, y_train, cv = cv, scoring = 'neg_mean_squared_error'))`

3.3 Evaluation Metrics: RMSE, MAE, R²

- **RMSE (Root Mean Squared Error):** Penalizes large errors heavily. Excellent for understanding worst-case scenarios and outlier sensitivity.
- **MAE (Mean Absolute Error):** Interpretable in the original units (e.g., Ksh). Represents the average absolute prediction error.
- **R² (R-squared):** The proportion of variance in the target variable explained by the model. (1.0 is perfect, 0.0 is a naive mean predictor).

Business Translation

"Our MAE is 979,664 Ksh, meaning on average, our valuation is off by roughly 1 million Ksh. Our R² of 0.645 means we explain 64.5% of the price variance."

3.4 Baseline Model CV Results

Model	Mean CV RMSE	Mean CV R ²
Ridge Regression	1,350,000	0.638
Lasso Regression	1,365,000	0.631
Elastic Net	1,352,000	0.637
Decision Tree	1,520,000	0.540
Random Forest	1,410,000	0.615
Gradient Boosting	1,390,000	0.625

Observation

Regularized linear models provide stable, well-calibrated baselines. Tree-based models capture non-linear thresholds but require tuning to prevent overfitting (higher variance).

Part 4: Hyperparameter Tuning

"Optimizing model complexity for peak performance."

4.1 The Need for Hyperparameter Tuning

- Default library parameters are rarely optimal for a specific dataset.
- **Underfitting (High Bias):** Model is too simple (e.g., Ridge `alpha` too high, Tree `max_depth` too low).
- **Overfitting (High Variance):** Model is too complex and memorizes noise (e.g., Ridge `alpha` too low, Tree `max_depth` too high).
- **Goal:** Find the "sweet spot" that minimizes validation error.

4.2 GridSearchCV for Ridge Regression

- We systematically test a grid of alpha values to find the optimal L2 penalty strength.

Python Implementation

```
1 from sklearn.model_selection import GridSearchCV
2
3 # Define the pipeline
4 ridge_pipe = Pipeline([('preprocessor', preprocessor),
5                         ('model', Ridge(random_state=42))])
6
7 # Define parameter grid (note the 'model__' prefix for pipeline steps)
8 ridge_param_grid = {
9     'model__alpha': [0.1, 1.0, 10.0, 100.0, 1000.0]
10 }
11
12 # Execute Grid Search
13 ridge_grid = GridSearchCV(ridge_pipe, ridge_param_grid, cv=5,
14                           scoring='neg_mean_squared_error', n_jobs=-1)
15 ridge_grid.fit(X_train, y_train)
16
```

4.3 GridSearchCV for Random Forest

- We tune tree depth and split criteria to balance predictive power with computational cost and overfitting risk.

Python Implementation

```
1 rf_pipe = Pipeline([( 'preprocessor', preprocessor),  
2                       ( 'model', RandomForestRegressor(random_state=42))])  
3  
4 rf_param_grid = {  
5     'model__n_estimators': [100, 200],  
6     'model__max_depth': [None, 10, 20],  
7     'model__min_samples_split': [2, 5]  
8 }  
9  
10 rf_grid = GridSearchCV(rf_pipe, rf_param_grid, cv=5,  
11                        scoring='neg_mean_squared_error', n_jobs=-1)  
12 rf_grid.fit(X_train, y_train)  
13
```

Part 5: Final Model Evaluation

"Proving generalization on the hold-out test set."

5.1 Evaluating on the Hold-out Test Set

The Golden Rule of ML

The test set (X_{test} , y_{test}) must remain **completely untouched** during all training, preprocessing fit, and hyperparameter tuning phases.

- **Step 1:** Select the best model from GridSearchCV (`best_estimator_`).
- **Step 2:** The pipeline automatically applies the learned preprocessing to X_{test} .
- **Step 3:** Generate predictions and calculate final, unbiased metrics (RMSE, MAE, R^2).

5.2 Final Model Ranking & Comparison

Model	Test RMSE	Test MAE	Test R ²
Tuned Ridge	1,338,647	979,664	0.645
Tuned Random Forest	1,355,210	995,120	0.636
Gradient Boosting	1,342,800	982,450	0.643

Interpretation

The Tuned Ridge model achieves the best balance of performance and simplicity (highest R², lowest RMSE). The complex tree-based models did not significantly outperform the regularized linear model, adhering to the principle of parsimony.

5.3 Interpreting the Metrics (Business Context)

- **$R^2 = 0.645$:** The model explains 64.5% of the variance in house prices. The remaining 35.5% is due to unobserved factors (e.g., neighborhood aesthetics, exact condition, market timing).
- **MAE = 979,664 Ksh:** On average, our valuation is off by roughly 1 million Ksh.
- **Business Decision:** Is this error margin acceptable? For a high-value property (e.g., 50M Ksh), a 1M Ksh error (2%) is highly acceptable for an automated valuation model (AVM). For a low-value property, the relative error is higher, suggesting a need for segment-specific models.

Part 6: Model Interpretability

"Opening the black box for stakeholder trust."

6.1 Why Interpretability Matters in Regression

- **Regulatory Compliance:** Financial institutions (e.g., mortgage lenders) must explain why a property was valued at a specific price.
- **Stakeholder Trust:** Real estate appraisers and clients are more likely to adopt a tool if they understand its logic.
- **Debugging:** Identifying if the model is relying on spurious correlations (e.g., using a proxy for a protected class).

Approach

While linear models offer direct coefficients, tree-based models offer **Feature Importances** (e.g., Gini importance), which we will extract and visualize.

6.2 Extracting Feature Importances

Python Implementation

```
1 # 1. Transform training data to get the exact feature names post-encoding
2 X_train_processed = best_model.named_steps['preprocessor'].transform(X_train)
3
4 # 2. Get feature names (numeric + one-hot encoded categorical)
5 feature_names = (
6     numeric_features +
7     list(best_model.named_steps['preprocessor']
8         .named_transformers_['cat']
9         .get_feature_names_out(categorical_features))
10 )
11
12 # 3. Extract importances and sort
13 importances = best_model.named_steps['model'].feature_importances_
14 importance_df = pd.DataFrame({'Feature': feature_names, 'Importance': importances})
15 importance_df = importance_df.sort_values(by='Importance', ascending=False)
16
```

6.3 Top Predictors for House Prices

- **Dominant Features:** area and bedrooms typically dominate real estate valuation, reflecting the fundamental principle that space drives value.
- **Secondary Features:** bathrooms, stories, and mainroad access provide measurable, albeit smaller, premiums.
- **Marginal Features:** Amenities like guestroom, basement, or hotwaterheating have minor impacts on the final price in this specific market.

Visualization

A horizontal bar chart of the top 10 features provides an immediate, intuitive visual for non-technical stakeholders.

Part 7: Practical Application & Policy

"From code to business impact and governance." "From code to business impact and governance."

7.1 Business & Policy Context Summary

Practical Applications

- **Real Estate:** Automated Valuation Models (AVMs) for banks and property portals.
 - **Agriculture:** Crop yield prediction based on soil metrics, rainfall, and fertilizer use.
 - **Finance:** Revenue forecasting and customer lifetime value (CLV) estimation.
-
- The chosen model (Tuned Ridge) balances predictive accuracy with the transparency required for regulatory compliance and stakeholder trust.
 - Increased model complexity (e.g., deep neural networks) did not translate into improved performance, validating the use of a parsimonious approach.

7.2 Recommendations for Deployment

Actionable Directives for Health/Business Systems

- ➊ **Monitor for Data Drift:** Track the distribution of incoming features. A sudden shift in the housing market (e.g., interest rate hikes) will degrade model performance.
- ➋ **Periodic Retraining:** Establish a pipeline to retrain the model quarterly with new data to adapt to shifting market trends.
- ➌ **Bias Auditing:** Ensure the model does not systematically undervalue properties in specific geographic or demographic regions.
- ➍ **Human-in-the-Loop:** The model should serve as a *decision support* tool for appulators, not an autonomous, final authority.

Questions?

Victor Wandera Lumumba

Statistician / Data Analyst / ML Expert

Mediacrest Training College

Nairobi, Kenya

Email: lumumbavictor172@gmail.com

Web: beyonddataanalytics.online

GitHub: [Lumumba1992](https://github.com/Lumumba1992)